Robust Convolutional Neural Network for Forest Fire Detection

H. Wharton
Email: zcaphwh@ucl.ac.uk

Department of Physics & Astronomy
University College London
London, United Kingdom, WC1E 6BT

Abstract:
Forest fires, a subset of wildfires, are rapidly spreading, often completely uncontrollable blazes that wreak havoc across the globe every annum. They are becoming more frequent and more destructive, directly contributing to climate change and furthering economic disparity. This paper presents a robust neural network ready for fire detection in woodland landscapes which can be implemented into early detection systems. The model is lightweight with 305,000 parameters for a total of 1.16MB in size. It can be deployed in junction with unmanned aerial vehicles (UAV) or closed-circuit television (CCTV). The model is designed to be resilient to environmental changes and technical failures. It features a convolutional neural network (CNN) architecture with two modes: (1) binary fire/no-fire classification, and (2) fire, no-fire, and lake classification, with heavy data augmentation to enhance resilience. This paper also presents the *Robust* metric for a network, indicating the level of resilience in the network to atypical variations in input data. Tested on unseen data, the binary model achieved a 0.86 F1 score and *Robust* of 0.69, whilst the ternary model achieved 81% accuracy overall.

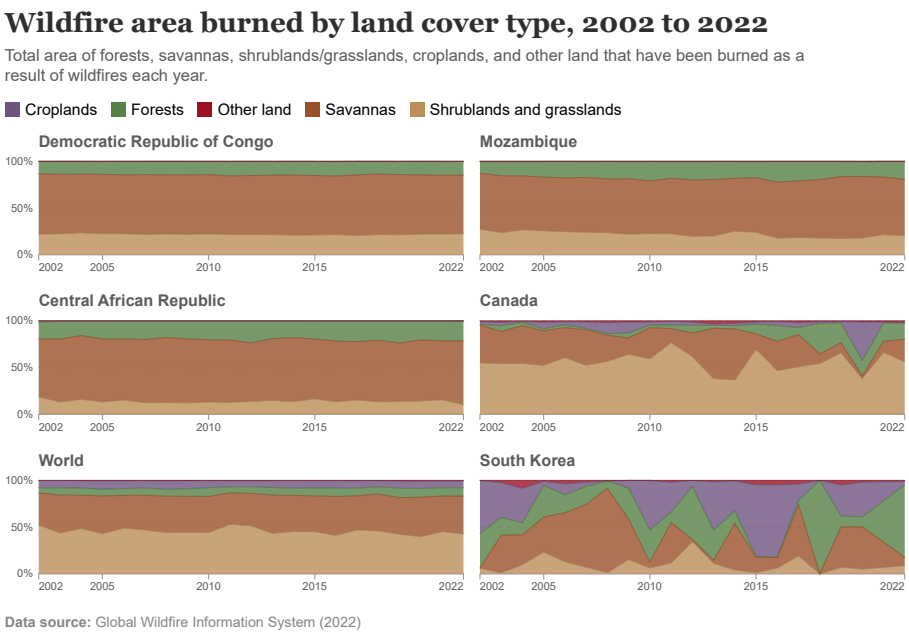

Figure 1: Data from OWID [1] showing the increase in forests as a percentage of area burnt during wildfires.

1 Introduction

1.1 Trends in Wildfire Incidence and Impact

Statistics from The Global Wildfire Information System (GWIS) [2] show a ∼25% decrease in the global area burnt due to wildfires since 2002. At a glance, one might consider this a success of the international effort to prevent the spread of wildfires. Since there is almost never a single, clearly identifiable starting location for large-scale fires, efforts have mostly gone into improving weather predictions, land management, education, and a particular focus on stricter enforcement of local laws around recreational fires and industrial activities in hotter, drier seasons [3]. These measures have evidently had some effect, however the decrease in total burnt area must be looked at with more nuance.

According to GWIS [2], burnt shrubland and grassland area on average per year has fallen by 101 million hectares or −42% between 2002 and 2022. This has been directly linked to a shift in agricultural management [4] in the developing world, especially in sub-Saharan Africa, where vast areas of shrublands have been cleared for new farmland to support rapidly growing populations. These cleared areas used to be hotspots for wildfires during summer months and would often be purposefully burnt off to control their growth; however, their permanent clearing has now led to a consistent decrease in annual burnt area.

On the other hand, GWIS [2] reports that annual land burnt due to forest fires has increased by 5 million hectares, or +20% since 2002. This fact can easily be overlooked when considering the overall decrease in burnt land; however, it is clear that in the past decades, forest fires have become more frequent. Shockingly, forest fires are not only becoming larger and more frequent, but they are also becoming more deadly [5], with estimates suggesting that over 10,000 additional deaths occurred each year in the 2010s than in the 1960s due to toxic particles in forest fire smoke.

In 2023, Canada saw some of it's worst forest fires on record, with 15 million hectares of forest burnt according to the Global Forest Watch [6], which is nearly 4% of Canada's total forested area. Sadly, 8 people lost their lives during the fires, with around 200,000 people being evacuated from their homes [7]. A single week of forest firest in June 2023 was estimated to have caused over $1.2 Billion CA in health impacts [8], further outlining the devastating impacts on human life. The

2023 fires continued to burn and eventually led to the 2024 fires, which sadly saw another 5 million hectares burnt and the tragic death of a firemen, who died whilst bravely tackling a blaze.

Studies have now also show a proven link between increased carbon emissions due to forest fires and climate change [9], with regions already susceptible to forest firest becoming increasingly drier and hotter. This is of course having adverse effects on nature, such as in the 2020 Australian fires, where an estimated 3 billion animals were killed [10].

Machine learning techniques have been a core part of the effort to prevent the spread of wildfires for over a decade. Successful attempts have looked at using satellite imagery for early detection [11], and efficient deep learning method for detecting active forest fires [12].

1.2 Neural Networks

A Neural Network (NN) is a general term, describing an interconnected network of nodes that processes information, this includes biological NNs, such as our brains, and also Artificial Neural Networks (ANNs) which are computational models inspired by the brain, but composed of artificial neurons organised in layers.

Seminal work was done by Turing in the 1950s, probing the mere possibility of a sequence of computations being able to learn and use information [13]. This theoretical foundation was developed by Rosenblatt in the 1960s, with the invention of the Perceptron [14], the first computationally possible neuron mimicking such in the brain. In the following decades, international and interdisciplinary efforts led to the development a vast subset of theoretical ANNs to perform tasks in computer vision, robotics, unsupervised learning, natural language processing and many, many more.

Deployment of such models was largely hindered by the lack of computational power, however with the advent of modern Graphics Processing Units (GPUs) in the early 2000s, the theory has become reality. NNs are widespread in science, industry and governance; estimates project a 37.3% annual compound growth of the global Artificial Intelligence (AI) market from 2024 to 2030 [15].

In 2024 the Nobel Prize in Chemistry was awarded to a group of scientists who used AI models to predict the 3D structure of proteins with a 98% accuracy [16], highlighting the rapid achievements that have been made.

1.3 Dataset description

The FLAME dataset [17] contains 47,992 frames taken directly from UAV footage recorded in Northern Arizona, USA in 2021. The footage was filmed in a woodland area during a prescribed pile burns, controlled fires used to reduce the risk of wildfire spread by creating natural barriers in the path of spreading fires.

39,375 images, labelled as "Fire" or "No Fire" are used for training with a further 8,617 images of the same classes used for testing. These two classes are clearly structured in directories, however there is another class "Lake" contained within the "No Fire" directory. This extra class is differentiated via image naming, however there were many cases of mislabelled data. If mislabelling is not rectified, it can corrupt training due to class imbalance and incorrect feature extraction.

1.4 Aims & Objectives

The aim of this work was to develop binary and ternary classifiers that can accurately label images of woodland scenery in the FLAME dataset [17] (and beyond) as belonging to one of the classes: Fire, No Fire or Lake. The Lake class can be thought of more generally as any substantial body of water, but for the remainder of this paper is referred to as Lake. The secondary aim of this work was to build in a high level of robustness in the model to a range of scenarios during data collection which might impact image properties such as orientation, pixel quality and more. The objectives met towards achieving these aims were: develop the network architecture, prepare the data for training, train the network, evaluate the model and simulate the aforementioned changes in image properties.

1.5 Summary of Paper

Section 2 details the methodology used, exploring: dataset restructuring, CNN design, data processing and augmentation, training metrics and consideration performance enhancement (e.g RGB channel filtering and gray-scaling). Section 2 will close with details of robustness simulations attempted after model training.

Training and testing results will be presented in Section 3 with due discussions of results in Section 4 and deployment Section 5. Final conclusion are found in Section 6.

2 Methodology

2.1 Dataset Structuring

The complete dataset is contained within a `.zip` file, with a structure outline in Figure 2. This binary structure is compatible with TensorFlow [18], the ML package used in this work, for the binary classification of "Fire" vs "No_Fire". Each final directory contains individual frames as (254x254x3) `.jpg` files.

```
FireNoFire
├── Training
│   ├── Fire
│   └── No_Fire
└── Test
    ├── Fire
    └── No_Fire
```

Figure 2: Diagram of directory structure for binary classification.

Note that for the ternary classification of "Fire" vs "Lake" or "No_Fire", restructuring was performed to achieve the structure outlined in Figure 3.

```
FireNoFire
├── Training
│   ├── Fire
│   ├── Lake
│   └── No_Fire
└── Test
    ├── Fire
    ├── Lake
    └── No_Fire
```

Figure 3: Diagram of directory structure for ternary classification.

Upon inspection of the dataset, the following issues were discovered:

1. Images labelled as `"lake_resized_lake_frame#.jpg"` in Training/No_Fire, frequently contained no lake at all, it turned out 54.5% of these images were mislabelled.

2. The test data contained 0 lake images; the binary classification model would learn to recognise lakes despite the testing data not containing any. Moreover, the ternary classification would require manual distribution of lake images into the testing data.

Both issues could be solved in parallel by manually creating separate versions of the structures in Figure 2 and 3 whilst distributing the mislabelled data. It is crucial in computer vision tasks that the model has zero access to testing data during training, it must remain entirely unseen to perform an unbiased evaluation of the model. Therefore, great caution must be taken if data is to be moved from the testing data. To distribute lake images into the testing dataset, a comprehensive list of all lake images needed to be created, for example `"lake_resized_lake_framei.jpg"` to `"lake_resized_lake_framej.jpg"` where all images within the slice [i,j] are the same lake. Once this list was created, all slices containing the same scenery (either lake or forest due to mislabelling) were grouped before being distributed to training or testing datasets. This systematic and cautious approach ensured minimum risk of data leakage.

It turns out that the reason for the mislabelling was because the UAV was randomly turned to face woodland area whilst filming different bodies of water around the capture area.

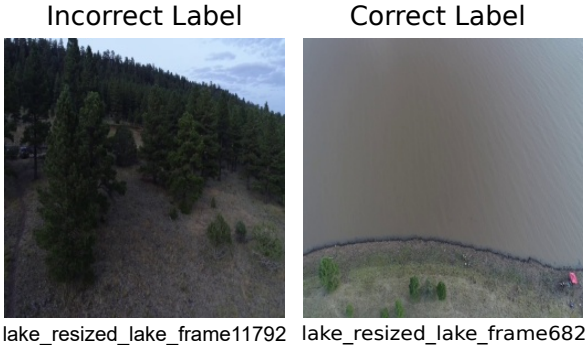

Figure 4: Example of correct vs incorrect labelling of lake data in raw training dataset.

The training data was further split into 80% training, 20% validation datasets. The validation is used to evaluate performance during the training process, aiding the fitting process.

Before the model could be created and trained, it was imperative to investigate the class balance of the training datasets. Imbalanced datasets can be mitigated during training, but if this isn't done, the model is expected to overfit to a single class and have a strong bias when predicting on unseen data.

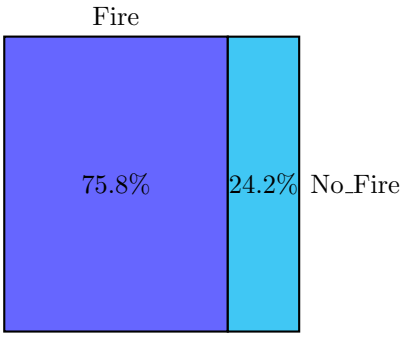

Figure 5: Distribution of training & validation data for binary classification. There is clearly a significant imbalance requiring mitigation to prevent bias in the model.

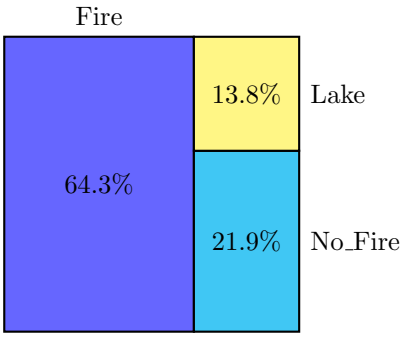

Figure 6: Distribution of training & validation data for the ternary classification. Again, there is a large imbalance which requires rectification.

There were two possible solutions to the imbalance that were considered. The first was to use augmentation, i.e. random operations that alter the qualities of the images, to systematically generate "new" images for the minority classes. Augmented images can be considered as being effectively "new" images because their properties such as, rotation, intensities, brightness and perspective can be changed. The more elegant solution which was used instead was to add a class weighting to the data before training. As an example for the binary classification, the weight for class 0 is calculated as:

$$w_0 = \frac{1}{n_{\text{neg}}} \times \frac{N}{2} \qquad (1)$$

where n_{neg} is the number of samples in class 0, and N is the total number of samples.

The weight for class 1 is calculated as:

$$w_1 = \frac{1}{n_{\text{pos}}} \times \frac{N}{2} \qquad (2)$$

where n_{pos} is the number of samples in class 1, and N is the total number of samples.

2.2 CNN Architecture

Convolutional Neural Networks (CNNs) are deep learning ANNs commonly used in computer vision tasks, they extract features positively associated with given classes and use these to make predictions about previously unseen images. Like any ANN, CNNs contain an input layer, some hidden layers and an output layer. CNNs work by reducing the dimensionality of the input image, whilst extracting the most important features as guided by a loss function. The loss function measures the effective difference between predicted output and actual target values.

The first layer of the network is a convolutional layer, which scans an $n \times n$ kernel across the input image, which is just a matrix-like grid of values. The dot product is taken between kernel weighting and the particular patch of the input image in focus. This allows the network to extract information about the features across the entire image.

The next layer is known as a pooling layer, which reduces the dimensionality of the network by extracting specified information from each kernel scanned across the grid. The two most common methods are to take either the max value or take an average of all values in a $n \times n$ pool. This layer increases training efficiency by constraining the network.

The convolutional and pooling layers are used sequentially, before the data is flattened into a 1D array and inputted to a dense layer. The dense layer used in the model has 32 nodes and is connected to a final layer with 1 output for the binary classification and 3 for the ternary.

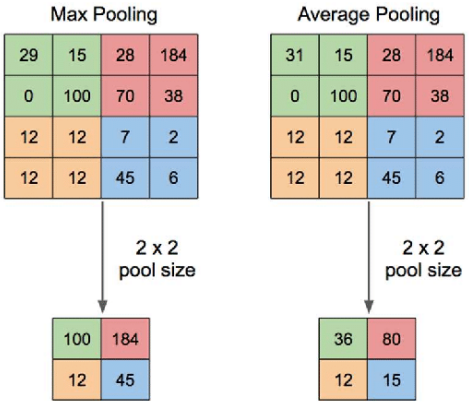

Figure 7: Example diagram of pooling layer in CNN.

Each layer has a non-linear activation applied the end, `LeakyReLu` (Leaky Rectified Linear Unit), developed by Mass et al [19].

The Leaky ReLU function is mathematically defined as:

$$f(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{if } x < 0 \end{cases} \quad (3)$$

Where x is the input to the neuron and α is a small constant, typically a value like 0.01, which determines the slope for negative values.

This function helps avoid the "dying ReLU" problem [20] by allowing small negative gradients during training.

Layer Type	Output Shape	Activation Function
Input	(254, 254, 3)	-
Rescaling	(254, 254, 3)	-
Conv2D (32 filters)	(254, 254, 32)	-
LeakyReLU	(254, 254, 32)	LeakyReLU
BatchNormalization	(254, 254, 32)	-
MaxPooling2D	(127, 127, 32)	-
Conv2D (64 filters)	(127, 127, 64)	-
LeakyReLU	(127, 127, 64)	LeakyReLU
BatchNormalization	(127, 127, 64)	-
MaxPooling2D	(63, 63, 64)	-
Conv2D (64 filters)	(63, 63, 64)	-
LeakyReLU	(63, 63, 64)	LeakyReLU
BatchNormalization	(63, 63, 64)	-
MaxPooling2D	(31, 31, 64)	-
Conv2D (32 filters)	(31, 31, 32)	-
LeakyReLU	(31, 31, 32)	LeakyReLU
BatchNormalization	(31, 31, 32)	-
MaxPooling2D	(15, 15, 32)	-
Flatten	(7200)	-
Dense (32 units)	(32)	-
LeakyReLU	(32)	LeakyReLU
Dropout (0.2)	(32)	-
Dense (1 unit)	(1)	Sigmoid

Table 1: CNN Model Architecture for the binary classification developed in `binary_model_making.ipynb` available on GitHub[1]. For the ternary classification, the final dense layer has 3 outputs and uses a softmax function which gives the probabilities of predictions. The mode has 305,000 parameters for a total size of 1.16MB.

An activation function in a NN is a mathematical function that determines the output of a neuron. It introduces non-linearity into the model, allowing the network to learn complex patterns and representations. Without activation functions, the network would essentially be a linear regression model, unable to capture intricate relationships in the data. Mathematically, an activation function f takes the weighted sum of inputs z (from the previous layer) and transforms it as $a = f(z)$ where: $z = w \cdot x + b$ (4) is the input to the neuron, with w being weights, x the input, and b the bias. a is the output of the neuron.

[1]GitHub page: `https://github.com/hug0-w/FireDetection/tree/main/ModelMaking`

To prevent overfitting, where a model learns the training data too well, including irrelevant details and noise, `Dropout` and `BatchNormalization` layers are added to the CNN. Dropout was developed by Srivastava and Hinton et al [21] as an elegant solution to the overfitting problem. When placed in the network between two layers, a specified percentage of weights will be forgotten or deactivated each iteration, in theory preventing the model from learning non-essential information over a long range of training steps (epochs). A 20% dropout is applied after the final dense layer. Batch normalisation was developed by Ioffe and Szegedy [22] and is a technique that normalises the inputs of each layer it is assigned to in the network. This helps when there is large differences between the intensities of features extracted from the image, which can lead to the exploding gradient problem [23], thereby reducing the risk of more subtle features not being learnt by the model.

Table 1 shows the final CNN that was implemented for training. For the binary task, the loss function is binary cross entropy and the ternary model uses sparse categorical entropy which are effective, modern loss functions for classification tasks [24]. Both models use Adam [25], a stochastic optimizer for gradient descent back propagation through the network, which determines the direction of the weights to push the loss towards the global minimum of the high-dimensional surface it exists on. The standard training rate of 0.001 was used, determining the size of each training step.

2.3 Preprocessing & Augmentation

To reduce training time and computational cost, the training dataset was preprocessed into batches of 64 images, large enough so that reasonable representation of each class is present over each batch. The training data was shuffled which prevents overfitting by effectively giving the a different training set each epoch (training over all batches once). The data was cached on the GPU system Random Access Memory (RAM), meaning that it isn't reloaded for each epoch but can be prepared in the RAM each time. Prefetching was also enabled, which means that during the n^{th} epoch, the $n^{th}+1$ training data is loaded in the background. These measures aim to ensure shorter training times, reducing the use of compute and energy cost.

Augmentation was applied to the training data for two purposes. Firstly, to prevent overfitting; since each time the training data is called it has random operations applied to it, which alter its properties and allow the model to learn the most general patterns with the overall effect of reducing the model's specificity to the exact conditions present in the raw images. Secondly, because the aim of this work was to develop a model that would be robust to certain changes to the input images, such as wind knocking the camera to the side or equipment faults; hence, these were simulated by augmentation. More discussion of the exact conditions the model was trained to be resilient to will be detailed in Section 2.5, but Figure 8 shows the random operations applied before training.

Figure 8: The left plots shows raw training images, the right plots are post-augmentation.

Some alternative approaches that aimed to enhance efficiency and model performance were to grayscale the images, and then to take only the red channel from the RGB channels.

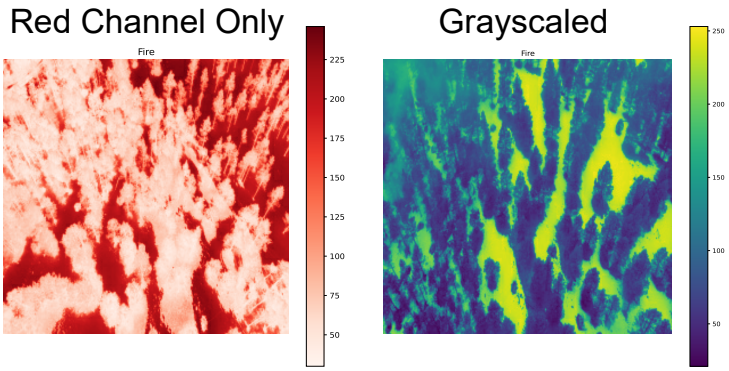

Figure 9: Training images with only red channel data and grayscaling applied.

Grayscaling the image still maintains information from each RGB channel, condensesing it into a single intensity. This will reduce the number of operations during training by a factor of 66.7%; reducing the computational cost and time during training. It remained unclear before training whether the model would perform similarly to when all RGB channels are present, because the model may turn out to have less access to more nuanced variations in the data.

To convert an RGB image to grayscale, the luminance formula from the ITU-R BT.601 standard was used [26]:

$$Y = 0.2989 \times R + 0.5870 \times G + 0.1140 \times B \quad (5)$$

Where R, G, B are the red, green, and blue colour channel values of a pixel and Y represents the grayscale intensity of that pixel.

As seen in Figure 9, the red channel values were extracted from the raw images. The theory behind this is that since wood and other flora typically burns at temperatures corresponding to red, orange and yellow flames, perhaps there is enough information in this channel only for the model to accurately make predictions. Immediately, there was a barrier to this approach; being that any white colour in the images, for example the sky or snow, would have very high intensities of red, which might affect the learning capability of the model. Considerations to overcome this included filtering out white from the images or reducing the overall brightness. Unfortunately, due to time constraints only limited work was done to investigate this.

This discussion lays the theoretical foundations for further investigation into exploring the possibility of tuning the RGB channels to aid the model in training.

Resizing, including both downscaling and upscaling, was performed before training to assess its impact on model performance.

2.4 Training

Metrics are calculated values which track the performance of the models during and after training.

Precision is the proportion of true positive predictions out of all positive predictions made.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (6)$$

Here, TP represents the number of true positives, and FP represents the number of false positives.

Recall measures the proportion of actual positive instances that were correctly predicted.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (7)$$

In this case, TP is the number of true positives, and FN is the number of false negatives. The F1 score is the harmonic mean of precision and recall, providing a single metric that balances both.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (8)$$

As seen in equation x, an F1 score of 1 indicates perfect predictions, while 0 means all predictions are incorrect.

Figure 10: Training metrics for the binary classification. Final weights available on GitHub.[2]

F1 score is useful metric for imbalanced datasets because of the balance between precision and recall. The binary models were trained using F1 score as their metric, where as the ternary model used accuracy (as a percentage).

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + FN} \tag{9}$$

The models trained on RGB, greyscale and red channels for the binary classifications were trained for 100 epochs, as was the ternary model which was tested only on RGB data.

Figures 10 and 11 both show the effects of dropout on the model during training, as the validation F1 Score or accuracy drops by a significant amount as weights are zeroed before rapidly returning to a stable solution, often with higher values than previously. If the model were to overfit, the validation F1 score and accuracy would saturate as training accuracy continued to increase; evidently this has been avoided due to a combination of dropout.

Model	Training Time (s)
Binary - RGB	1400
Binary - Grayscale	1000
Binary - Red	1000
Binary - Resized 200x200	700
Binary - Resized 300x300	1900
Ternary - RGB	1600

Table 2: Total training time over 100 epochs for different model types.

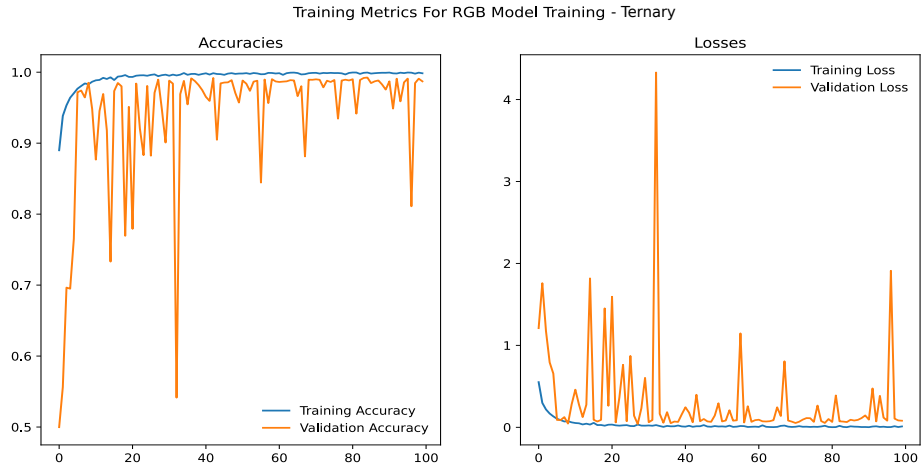

Figure 11: Training metrics for the ternary classification. It is possible that minor overfitting has occurred as the validation accuracy failed to reach training accuracy.

The observed decrease (or increase) in training time, seen in Table 2, is explained by a change in the total number of operations per epoch; grayscaling the image reduces the dimensions from (254x254x3) to (254x254x1) and similarly downscaling reduced it to (200x200x3). Section 3 will present the evaluation results.

2.5 Robustness Simulations

The US National Weather Service has outlined a direct link between strong winds and the rapid spread of woodland fires [27]. The devastating 2024 Los Angeles wildfires have been linked to the Santa Ana winds, which blew up to 160 kmh^{-1} ($44ms^{-1}$) for days on end as measured by the ABC Climate unit [28]. Deploying a small, fragile UAV or drone in such winds is practically impossible, but it is during the build up of such gusts where the drone would be deployed. As these gusts develop, the drone would be bombarded with forces, potentially knocking it over or worse, shifting its camera to a fixed off-centred position. This could be mitigated with a gyroscopic camera mount, however this is a very expensive solution. What this work proposes is that by simulating such winds in the training data, simply by randomly rotating and flipping images, the model can learn to detect fire regardless of the orientation.

Another issue is glare, where bright streaks of light enter the lens during capture. This can be simulated with random increases of brightness. There is also the possibility of capture failure, simulated by random decreases in brightness. Finally, it is also possible that power failure could occur during flight, which may result in a loss of colour in frames where RGB was expected, this is simulated by random grayscale. Figure 12 shows examples images of affected frames. All augmentations were applied before training, and then each scenario was tested individually for the RGB model post training. For this work, only the binary model was tested for robustness.

A new *Robust* metric is defined as:

$$Robust = \frac{\text{Mean F1 score over N scenarios}}{\text{Base F1 Score}} \frac{N}{N+1} \quad (10)$$

Where values close to 1 indicate robustness in the model, and the last term rewards more scenarios being considered (up to a saturation).

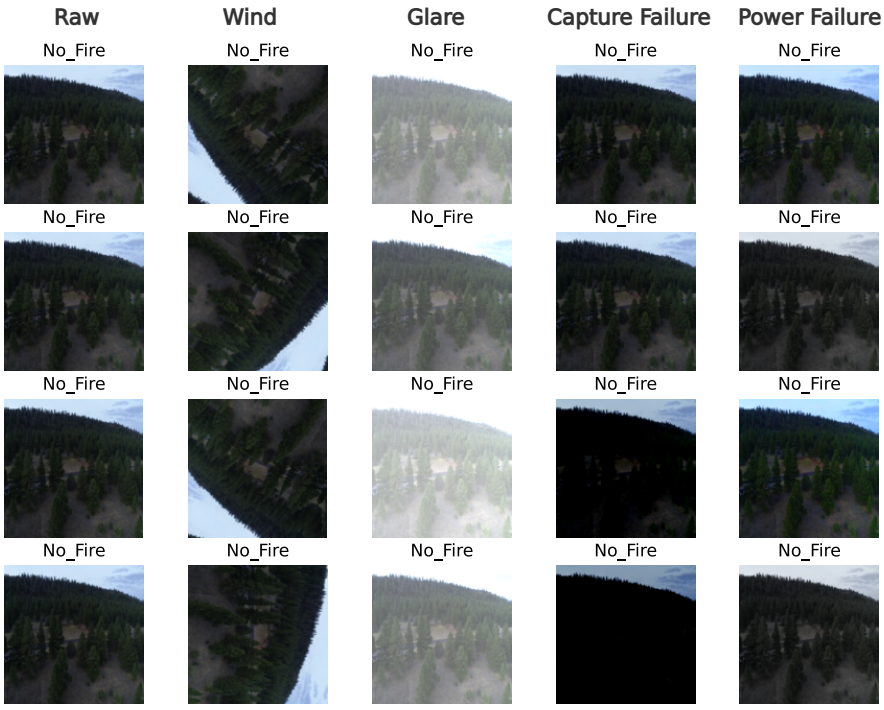

Figure 12: Example frames from different proposed scenarios which affect the quality of input during model deployment. Robustness is only tested for the binary classification.

[2]Weights available here GitHub page: `https://github.com/hug0-w/FireDetection/tree/main/Weights`

3 Results

Following 100 epochs of training, evaluations were made of each model. First the model was evaluated on the testing data using the relevant metric. This indicates how well the model performs on unseen data, such as images inputted when deployed into use. Confusion matrices were generated, which display the models predictions vs true labels. For binary classification, the threshold, x , is defined such that:

$$\text{if} \begin{cases} \text{Network Output} > x & , \text{ Prediction} = 1 \\ \text{Network Output} < x & , \text{ Prediction} = 0 \end{cases}$$

Receiver Operating Characteristic (ROC) curves were made to further evaluate the model, which plots the True Positive Rate TPR against the False Positive Rate FPR at different thresholds.

The Area Under the Curve (AUC) of the ROC curve is an evaluation metric which encapsulates how well the model is at sorting positive (1) and negative (0) predictions. An AUC of 1 would be a perfect score and a value of 0.5 is equivalent to making random guesses.

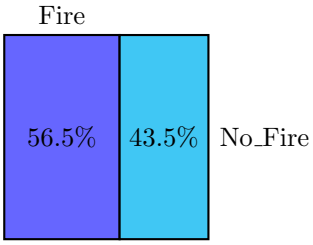

Figure 13: Distribution of testing data for binary classification. The classes are balanced, meaning the model can be tested on both classes evenly.

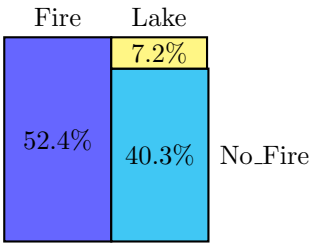

Figure 14: Distribution of testing data for binary classification. There is large imbalance; the model will be evaluated ∼5 times more for the Fire class than Lake.

This section will introduce the final results obtained with further discussion in Section 4.

3.1 Binary Classification

Model	F1 Score	Accuracy
Binary - RGB	**0.87**	-
Binary - Rs 200x200	0.80	-
Binary - Rs 300x300	0.72	-
Binary - Red	0.46	-
Binary - Grayscale	0.44	-
Ternary - RGB	-	**81%**

Table 3: Models ranked by evaluation metric after 100 epochs of training. For the binary task, the RGB model ranks 1st followed by the Resized (Rs) models and the red and grayscale models are last.

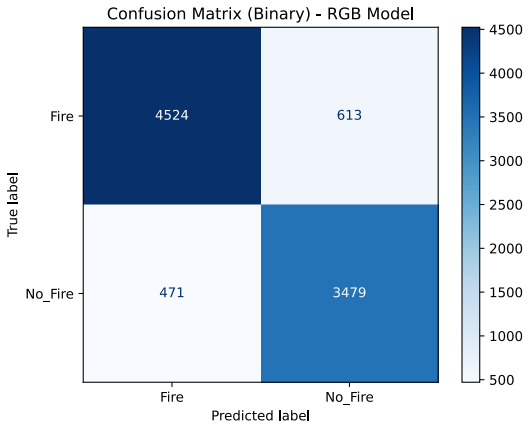

Figure 15: Confsuion matrix for RGB model in binary classification. The threshold is 0.5. The model performs well, correctly predicting 88% of test images. The slight bias toward predicting No_Fire may be due to the class weighting mentioned in Section 2.1.

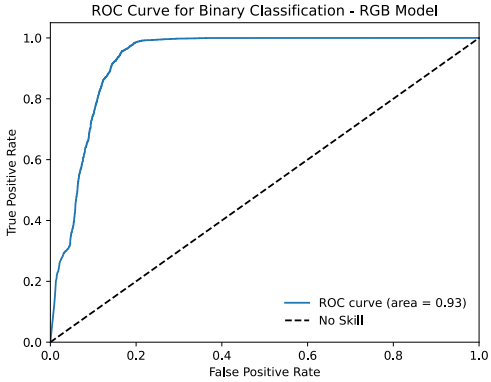

Figure 16: ROC curve for the RGB model in binary classification. The model performs exceptionally well with AUC = 0.93, close to 1 and much better than randomly guessing.

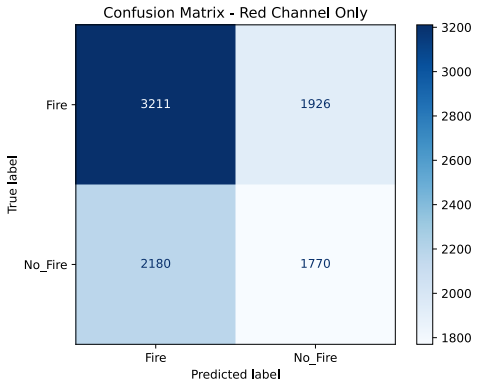

Figure 17: Confusion matrix for red channel model in binary classification with a threshold of 0.5. The matrix shows a bias towards predicting Fire, suggesting the model was missing information during training about the No Fire class.

3.2 Ternary Classification

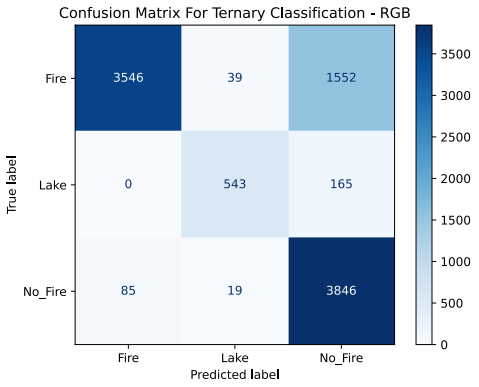

Figure 18: Confusion matrix for the ternary classification. The model performs exceptionally well for the No Fire and Lake classes, suggesting some strong generalisation in the model. The Fire class is often mislabelled as No Fire, a flaw discussed in section 4.

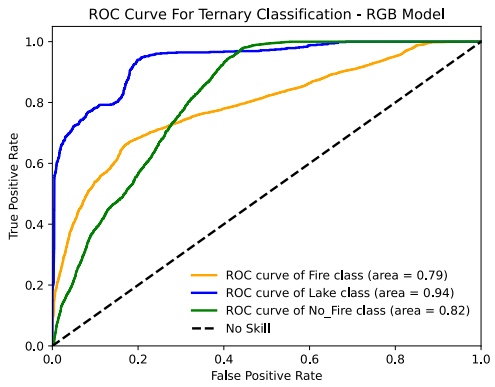

Figure 19: ROC curve for the ternary classification. The Lake class scores ∼1, perahps due to the simple features associated with it (i.e. water).

3.3 Robustness Simulations

Scenario	F1 Score
Wind	**0.78**
Glare	0.75
Capture Failure	0.75
Power Failure	0.69
Robust	0.69

Table 4: Evaluation results for the scenarios simulated on the binary RGB model. The model is most robust to wind, scoring only 0.1 less than with no augmentation. The model performs similarly for glare and capture failure, which makes sense as they are equal and opposite effects. Simulated power failure causes the biggest drop in F1 score which was expected since the RGB model requires the data from all 3 colour channels to make its predictions. The *Robust* value (defined in Section 2.5) indicates a good degree of robustness in the model.

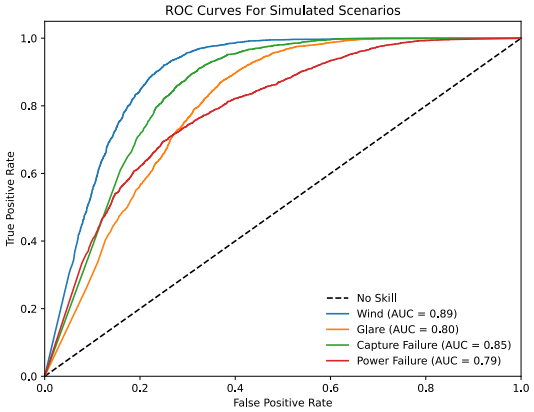

Figure 20: ROC curve for simulated scenarios on the RGB model. The same overall trend matches the confusion matrix, however interestingly, capture failure appears to perform better than glare, possibly because the glare introduces spurious data whereas the capture failure only removes data. Essentially glare makes it harder for the model to extract features than capture failure.

4 Discussion

The RGB CNN model developed performs excellently with a final F1 score of 0.86. This suggests that model extracted highly general features from training images that is recognised in the test data and used to make mostly correct predictions.

For the binary classification task, the threshold can be adjusted to factor in the level of risk associated with a FN vs a FP. In

other words, is it more risky for the model to incorrectly predict the No Fire class for an image containing a fire, or for the model to predict the Fire class when in fact the image does contain a fire. Clearly the latter has less serious consequences, so the threshold could be lowered to prevent the first case. In fact, by using a confusion matrix, the model could be tuned by hand on the test data until a harmonious balance is struck between over predicting and under predicting the Fire class. In practical terms this would mean that when the model is deployed, it doesn't raise a false alarm neither does it not raise it when it should.

The RGB model shows strong robustness to the possible issues during image collection; in the event of strong winds knocking the camera out of position, a high level of confidence can be maintained in the model. The ternary model performed well for the No Fire and Lake class, however it mislabelled 31% of test images containing fires. This is a large concern due to similar arguments made in the previous paragraphs about thresholds. Since the ternary model doesn't output a single threshold, but instead a predicted probability for each class, there isn't an elegant solution as in the binary case. The model simply lacks the generalisation and knowledge of the features to be able to make the multi-class prediction accurately. The model was trained with large class weights as calculated in equations (1) and (2), which perhaps need further tuning to get better results. Moreover, a simple solution might be to employ a more efficient CNN, or a large model, inevitably requiring more time and compute.

A large controversy in the use of machine learning techniques is the large use of compute power required to train the models. This model is only 1.16MB with 305,000 parameters, a far cry from the hundreds of billions of parameters used to train Large Language Models (LLMs) such as ChatGPT [29]. Training took less than a few minutes for each model thanks to TensorFlow [18] which enabled the preprocessing steps mentioned in Section 2.3.

5 Deployment

To deploy either the RGB binary or ternary model, model functions and weights are available on GitHub [3]. Other model weights are available upon request.

With the weights loaded, users can train the model on local data. Overfitting prevention is built-in, but for larger datasets, increasing dropout is recommended for better generalisation.

The model can be implemented using a single board computer chip such as Raspberry Pi [30] in parallel with a CCTV camera mounted above a woodland area. Or perhaps a weekly drone flyover could be performed (daily in summer), with up-to-date predictions made using a wireless transfer of the live footage.

6 Conclusion

This paper has covered the development of a neural network for the detection of forest fires. Considerations have been made into model generalisation and image augmentation, with various training and evaluation styles utilised.

The binary model performs exceptionally well for Fire vs No Fire classifications with an evaluation F1 score of 0.86, and *Robust* (see Section 2.5) score 0.69. The first metric implies accurate predictions can be made by the model and the latter indicates strong resilience in the model to atypical inputs.

With more development, the ternary model can be brought up to a similar level of performance; currently it has issues with the Fire class, and some potential improvements have been discussed in Section 4 such as tuning the class weighting. Future development might also include image segmentation, where specific pixels believed to contain fires are labelled by the model, which could provide more efficient locating of fires.

Ultimately, this work presents a novel and meaningful solution to assist the early detection of forest fires, with room for extra development made possible by easy accessibility to the model weights and files.

[3]For model deployment, visit: `https://github.com/hug0-w/FireDetection/tree/main/Deployment`

References

[1] V. Samborska and H. Ritchie, "Wildfires," *Our World in Data*, 2024, https://ourworldindata.org/wildfires.

[2] European Commission, *Seasonal trend analysis - gwis statistics portal*, Accessed: 2025-03-30, 2025. [Online]. Available: https://gwis.jrc.ec.europa.eu/apps/gwis.statistics/seasonaltrend.

[3] M. Carreiras, A. J. D. Ferreira, S. Valente, *et al.*, "Comparative analysis of policies to deal with wildfire risk," *Land Degradation & Development*, vol. 25, no. 1, pp. 92–103, 2014.

[4] T. Lasanta, M. Khorchani, F. Pérez-Cabello, P. Errea, R. Sáenz-Blanco, and E. Nadal-Romero, "Clearing shrubland and extensive livestock farming: Active prevention to control wildfires in the mediterranean mountains," *Journal of Environmental Management*, vol. 227, pp. 256–266, Dec. 2018. DOI: 10.1016/j.jenvman.2018.08.104.

[5] C. Y. Park, K. Takahashi, S. Fujimori, *et al.*, "Attributing human mortality from fire pm2.5 to climate change," *Nature Climate Change*, Oct. 2024. DOI: 10.1038/s41558-024-02149-1. [Online]. Available: https://www.nature.com/articles/s41558-024-02149-1.

[6] Global Forest Watch, *Global forest watch map*, Accessed: 2025-03-30, 2025. [Online]. Available: https://www.globalforestwatch.org/map.

[7] M. W. Jones, D. I. Kelley, C. A. Burton, *et al.*, "State of wildfires 2023–2024," *Earth System Science Data*, vol. 16, pp. 3601–3685, 2024. DOI: 10.5194/essd-16-3601-2024. [Online]. Available: https://essd.copernicus.org/articles/16/3601/2024/.

[8] M. Martin-Richon, *With canada's forest fires, the health costs hit home*, Canadian Climate Institute, Jun. 2023. [Online]. Available: https://climateinstitute.ca/with-the-forest-ablaze-the-health-costs-hit-home/.

[9] M. W. Jones, S. Veraverbeke, N. Andela, *et al.*, "Global rise in forest fire emissions linked to climate change in the extratropics," *Science*, vol. 386, no. 6719, eadl5889, 2024. DOI: 10.1126/science.adl5889. eprint: https://www.science.org/doi/pdf/10.1126/science.adl5889. [Online]. Available: https://www.science.org/doi/abs/10.1126/science.adl5889.

[10] R. Davare, "The impact of wildfires on biodiversity and the environment," *Earth.Org*, 2022, Accessed: 2025-03-30. [Online]. Available: https://earth.org/impact-of-wildfires/.

[11] R. S. priya and K. Vani, "Deep learning based forest fire classification and detection in satellite images," in *2019 11th International Conference on Advanced Computing (ICoAC)*, 2019, pp. 61–65. DOI: 10.1109/ICoAC48765.2019.246817.

[12] S. T. Seydi, V. Saeidi, B. Kalantar, N. Ueda, and A. A. Halin, "Fire-net: A deep learning framework for active forest fire detection," *Journal of Sensors*, vol. 2022, G. Pennazza, Ed., pp. 1–14, Feb. 2022. DOI: 10.1155/2022/8044390.

[13] A. M. Turing, "Computing machinery and intelligence," *Mind*, vol. LIX, no. 236, pp. 433–460, 1950. DOI: 10.1093/mind/LIX.236.433.

[14] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," Cornell Aeronautical Laboratory, Technical Report VG-1196-G-8, 1958. [Online]. Available: https://www.ling.upenn.edu/courses/cogs501/Rosenblatt1958.pdf.

[15] S. R. Department, *Global ai market size 2030*, Accessed: 2025-03-30, 2024. [Online]. Available: https://www.statista.com/forecasts/1474143/global-ai-market-size.

[16] J. Jumper, R. Evans, A. Pritzel, *et al.*, "Highly accurate protein structure prediction with alphafold," *Nature*, vol. 596, pp. 583–589, 2021. DOI: 10.

1038/s41586-021-03819-2. [Online]. Available: https://www.nature.com/articles/s41586-021-03819-2.

[17] A. Shamsoshoara, F. Afghah, A. Razi, L. Zheng, P. Fulé, and E. Blasch, *The flame dataset: Aerial imagery pile burn detection using drones (uavs)*, Accessed: 2025-03-30, 2021. [Online]. Available: https://ieee-dataport.org/open-access/flame-dataset-aerial-imagery-pile-burn-detection-using-drones-uavs.

[18] Martín Abadi, Ashish Agarwal, Paul Barham, *et al.*, *TensorFlow: Large-scale machine learning on heterogeneous systems*, Software available from tensorflow.org, 2015. [Online]. Available: https://www.tensorflow.org/.

[19] A. L. Maas, A. Y. Hannun, and A. Y. Ng, "Rectifier nonlinearities improve neural network acoustic models," in *Proceedings of the 30th International Conference on Machine Learning (ICML 2013)*, 2013. [Online]. Available: https://ai.stanford.edu/~amaas/papers/relu_hybrid_icml2013_final.pdf.

[20] S. C. Douglas and J. Yu, "Why relu units sometimes die: Analysis of single-unit error backpropagation in neural networks," in *2018 52nd Asilomar conference on signals, systems, and computers*, IEEE, 2018, pp. 864–868.

[21] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: http://jmlr.org/papers/v15/srivastava14a.html.

[22] S. Ioffe and C. Szegedy, *Batch normalization: Accelerating deep network training by reducing internal covariate shift*, 2015. arXiv: 1502.03167 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1502.03167.

[23] G. Philipp, D. Song, and J. G. Carbonell, "The exploding gradient problem demystified-definition, prevalence, impact, origin, tradeoffs, and solutions," *arXiv preprint arXiv:1712.05577*, 2017.

[24] J. Terven, D. Cordova-Esparza, A. Ramirez-Pedraza, and E. Chavez-Urbiola, *Loss functions and metrics in deep learning a preprint*, 2023. [Online]. Available: https://arxiv.org/pdf/2307.02694.

[25] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, 2017. arXiv: 1412.6980 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1412.6980.

[26] International Telecommunication Union, *Recommendation ITU-R BT.601: Studio Encoding Parameters of Digital Television for Standard 4:3 and Wide-Screen 16:9 Aspect Ratios*, [Online]. Available: https://www.itu.int/rec/R-REC-BT.601, 1982.

[27] National Weather Service, NOAA, *Fire weather: General winds*, Accessed: 2025-03-30, Access Year. [Online]. Available: https://www.weather.gov/media/zhu/ZHU_Training_Page/winds/FireWx_General_Winds/FireWx_General_Winds.pdf.

[28] A. C. Unit, *The science behind the la wildfires*, Accessed: 2025-03-30, 2025. [Online]. Available: https://6abc.com/post/science-behind-la-wildfires/15799728/.

[29] T. Brown, B. Mann, N. Ryder, *et al.*, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.

[30] W. Gay, *Raspberry Pi Hardware Reference*. Jan. 2014, ISBN: 978-1-4842-0800-7. DOI: 10.1007/978-1-4842-0799-4.